

Monte Carlo Tree Search for Operations Research: The Job Shop Scheduling Problem

1. Problem Statement

- **Domain:** Combinatorial Optimization and Resource Allocation
- **Problem:** The Job Shop Scheduling Problem (JSSP)

In a Job Shop environment, multiple unique jobs must be processed across a set of specialized machines. Each job consists of a strict sequence of operations, and each operation requires a specific machine for a predetermined duration. The environment is bound by strict physical constraints: a machine can only process one operation at a time, and an operation cannot begin until its preceding step is completed.

The primary objective is to minimize the Makespan, the total time elapsed from the initiation of the first operation to the completion of the final operation across all jobs.

JSSP is notoriously NP-hard due to the combinatorial explosion of the state space. As the number of jobs and machines increases, the number of possible scheduling permutations grows astronomically. Traditional greedy algorithms (such as "Shortest Processing Time First") are highly susceptible to local optima, frequently prioritizing short-term gains that ultimately create severe bottlenecks, leaving critical machines idle. Furthermore, designing an accurate static evaluation function to determine the "value" of a partially completed schedule is nearly impossible, rendering standard heuristic search methods ineffective.

2. How MCTS Solves the Problem

Monte Carlo Tree Search (MCTS) circumvents the need for a static heuristic evaluation by treating schedule generation as a sequential decision process. Rather than attempting to calculate the exact value of a mid-game board state, MCTS approximates the value of scheduling decisions through rapid, simulated rollouts. The algorithm incrementally builds a search tree where each Node represents a valid partial schedule, and each Edge represents the decision to assign an operation to a machine.

This is achieved through the continuous iteration of four distinct processes:

I. Selection (Strategic Navigation)

The algorithm traverses the existing search tree from the root (the current factory state) down to a leaf node using the Upper Confidence Bound applied to Trees (UCT) formula. The UCT formula balances Exploitation (selecting scheduling paths that have previously yielded highly efficient, low-makespan results) with Exploration (forcing the algorithm to test neglected sequences to ensure a better global optimum isn't missed).

II. Expansion (Constraint-Aware Growth)

Upon reaching a leaf node that is not a terminal state, the tree is expanded. The algorithm identifies all "Ready" operations—actions where the preceding job steps are finished and the required machine is currently idle. A new child node is created for one of these valid actions.

This constraint-aware expansion ensures the search tree only grows within the physical limitations of the factory floor, avoiding computationally wasteful invalid states.

III. Simulation (Heuristic-Guided Rollout)

From the newly expanded child node, the agent simulates the remainder of the factory day until all jobs are completed. To improve the signal-to-noise ratio of the simulation, this implementation utilizes an epsilon-greedy policy. Instead of purely random guessing, the simulation selects the job with the "Most Work Remaining" 80% of the time, and selects randomly 20% of the time. This provides a high-fidelity forecast of the node's true potential.

IV. Backpropagation (Value Informing Action)

Once the simulated schedule is complete, the final Makespan is converted into a continuous reward signal ($\text{Reward} = 1000 / \text{Makespan}$). This value is passed back up the precise path taken during the Selection phase. The visit count and average score of every node in that path are updated. Over thousands of iterations, the optimal starting sequences naturally emerge with the highest visit counts, allowing the algorithm to bypass bottlenecks.



Fig: Gantt Chart of MCTS optimization